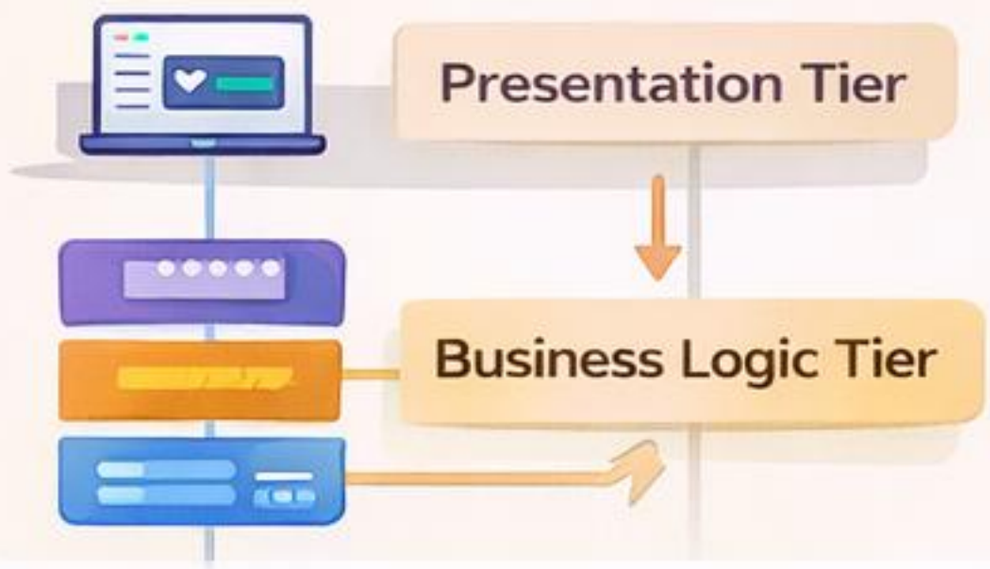
Day 4: Architecture, Messaging, and APIs

How services are organized and how they communicate inside distributed systems

N-Tier Architecture

N-tier architecture separates presentation, business logic, and data access



Publish-Subscribe

Subscribers listen to a 'topic" and are notified of relevant events

Monoliths vs. Microservices

Microservices divide a system into autonomous and independently deployble components

Message Broker

Message broker enables ddurenous communication between services

Event-Driven Architecture

Emit events to an event bus to drive real-time systems with decoupled services

API Gateway

API Gateway handles all client requests and routes them to internal services

Event-Driven Architecture

ESB centralizes. integration logic, but can become a bottlenek and d CORS

Real-Time Communication

Long polling, WebSockets, and Server-Sent Events (SSE) enable real-time, two-yinteraction

Choosing the right **communication model and service boundaries** keeps **systems scalable, maintainable, and responsive.**

Day 4: Architecture, Messaging, and APIs

Day 4 focuses on how services are organized and how they communicate inside distributed systems.

It covers:

- N-tier architecture and separation of responsibilities
- Message brokers, message queues, and publish-subscribe
- Enterprise Service Bus (ESB)
- Monoliths, modular monoliths, and microservices
- Event-Driven Architecture (EDA), event sourcing, and CQRS
- API Gateway and Backend For Frontend (BFF)
- REST, GraphQL, and gRPC
- Long polling, WebSockets, and Server-Sent Events (SSE)

In short, Day 4 is about choosing the right communication model and service boundaries so the system remains scalable, maintainable, and responsive.

Core ideas

N-tier architecture

- `N-tier architecture` separates presentation, business logic, and data access concerns.
- It improves maintainability and scalability by isolating responsibilities.
- More tiers usually improve separation, but they also add network hops and operational complexity.

Message brokers, queues, and publish-subscribe

- `Message brokers` help services communicate asynchronously without direct coupling.
- `Message queues` are point-to-point and are good for background jobs and work distribution.
- `Publish-subscribe` broadcasts events to many consumers and is useful for fan-out notifications and decoupled event handling.

ESB, monoliths, and microservices

- `ESB` centralizes integration logic, but it can become a bottleneck and a single point of failure.
- `Monoliths` are simple to start with and easier to debug early on.
- `Microservices` trade simplicity for team autonomy, independent scaling, and better fault isolation.
- A poorly designed microservices system can become a distributed monolith.

Event-Driven Architecture, event sourcing, and CQRS

- `EDA` uses events to communicate state changes between services.
- `Event sourcing` stores domain changes as an append-only log instead of only storing the latest state.
- `CQRS` separates write paths from read paths so each can scale and evolve differently.

API Gateway and API styles

- `API Gateway` provides a single entry point for clients and can handle authentication, routing, rate limiting, logging, and caching.
- `REST` is simple and widely supported.
- `GraphQL` reduces over-fetching and gives the client more control over response shape.
- `gRPC` is efficient for service-to-service communication and supports code generation and streaming.

Real-time communication

- `Long polling` works but is inefficient at scale.
- `WebSockets` provide full-duplex, persistent communication and are strong for chat and live updates.
- `SSE` is server-to-client only and is simpler than WebSockets for streaming updates.

Day-4 sample: Publish-Subscribe Event Bus

This repository includes a small publish-subscribe event bus sample in both Python and Java.

Functional requirements

- Allow services to subscribe to a topic
- Publish events to a topic
- Deliver every event to all subscribers of that topic
- Keep the API simple enough to explain in an interview

High-level components

- `Publisher`: emits an event
- `Event bus`: stores subscriptions and dispatches messages
- `Topic`: logical grouping for events
- `Subscribers`: services that react to the event independently

Data flow

1. Services subscribe handlers to a topic such as `order.created`
2. A publisher emits an event to that topic
3. The event bus fans the event out to all subscribed handlers
4. Each subscriber processes the event independently
5. New subscribers can be added without changing the publisher

Project structure

```
Day-4/
  README.md
  pub-sub-sample/
    python/
      event_bus.py
      demo.py
```

```
    java/
      EventBus.java
      Main.java
```

Run the sample

Python

```
cd pub-sub-sample\python
python demo.py
```

Java

```
cd pub-sub-sample\java
javac Main.java EventBus.java
java Main
```

Interview takeaways

- Service boundaries should follow the business domain, not just technical layers
- Async messaging improves decoupling, but delivery guarantees and ordering become important
- API Gateway is useful, but it must not turn into a bottleneck
- REST, GraphQL, and gRPC solve different problems; pick based on client needs and latency profile
- WebSockets are usually the right choice when two-way, low-latency communication matters

Next improvements

- Add unsubscribe support
- Add retry and dead-letter queue simulation
- Add event persistence for replay
- Add message filtering and wildcard topics
- Extend the sample into an event-driven order processing workflow

Source Code Appendix

Day-4/pub-sub-sample/python/event_bus.py

```
from __future__ import annotations

from collections import defaultdict
from dataclasses import dataclass, field
from threading import Lock
from typing import Callable


Handler = Callable[[str, dict], None]


@dataclass
class EventBus:
    _subscribers: dict[str, list[Handler]] = field(default_factory=lambda: defaultdict(list), init=False)
    _lock: Lock = field(default_factory=Lock, init=False)

    def subscribe(self, topic: str, handler: Handler) -> None:
        if not topic:
            raise ValueError("Topic must not be empty")

        with self._lock:
            self._subscribers[topic].append(handler)

    def publish(self, topic: str, payload: dict) -> int:
        with self._lock:
            handlers = list(self._subscribers.get(topic, []))

        for handler in handlers:
            handler(topic, payload)

        return len(handlers)

    def topic_snapshot(self) -> dict[str, int]:
        with self._lock:
            return {topic: len(handlers) for topic, handlers in self._subscribers.items()}
```

Day-4/pub-sub-sample/python/demo.py

```
from event_bus import EventBus


def notification_service(topic: str, payload: dict) -> None:
    print(f"[notification-service] topic={topic} payload={payload}")


def analytics_service(topic: str, payload: dict) -> None:
    print(f"[analytics-service] topic={topic} payload={payload}")


def inventory_service(topic: str, payload: dict) -> None:
    print(f"[inventory-service] topic={topic} payload={payload}")


def main() -> None:
    bus = EventBus()

    bus.subscribe("order.created", notification_service)
    bus.subscribe("order.created", analytics_service)
    bus.subscribe("order.created", inventory_service)
    bus.subscribe("user.signed_up", analytics_service)

    print("Topic snapshot:", bus.topic_snapshot())

    delivered = bus.publish(
        "order.created",
        {"order_id": "ORD-1001", "user_id": "U-42", "amount": 1499},
    )
    print(f"Delivered to {delivered} subscribers")

    delivered = bus.publish(
        "user.signed_up",
        {"user_id": "U-99", "plan": "free"},
    )
    print(f"Delivered to {delivered} subscribers")


if __name__ == "__main__":
    main()
```

Day-4/pub-sub-sample/java/EventBus.java

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

public class EventBus {
    @FunctionalInterface
    public interface EventHandler {
        void handle(String topic, Map<String, Object> payload);
    }

    private final Map<String, List<EventHandler>> subscribers;

    public EventBus() {
        this.subscribers = new HashMap<>();
    }

    public synchronized void subscribe(String topic, EventHandler handler) {
        if (topic == null || topic.isBlank()) {
            throw new IllegalArgumentException("Topic must not be empty");
        }

        subscribers.computeIfAbsent(topic, key -> new ArrayList<>()).add(handler);
    }

    public int publish(String topic, Map<String, Object> payload) {
        List<EventHandler> handlers;
        synchronized (this) {
            handlers = new ArrayList<>(subscribers.getOrDefault(topic, List.of()));
        }

        for (EventHandler handler : handlers) {
            handler.handle(topic, payload);
        }

        return handlers.size();
    }

    public synchronized Map<String, Integer> topicSnapshot() {
        Map<String, Integer> snapshot = new LinkedHashMap<>();
        for (Map.Entry<String, List<EventHandler>> entry : subscribers.entrySet()) {
            snapshot.put(entry.getKey(), entry.getValue().size());
        }
        return snapshot;
    }
}
```

Day-4/pub-sub-sample/java/Main.java

```
import java.util.LinkedHashMap;
import java.util.Map;

public class Main {
    public static void main(String[] args) {
        EventBus bus = new EventBus();

        bus.subscribe("order.created", (topic, payload) ->
                System.out.println("[notification-service] topic=" + topic + " payload=" + payload));
        bus.subscribe("order.created", (topic, payload) ->
                System.out.println("[analytics-service] topic=" + topic + " payload=" + payload));
        bus.subscribe("order.created", (topic, payload) ->
                System.out.println("[inventory-service] topic=" + topic + " payload=" + payload));
        bus.subscribe("user.signed_up", (topic, payload) ->
                System.out.println("[analytics-service] topic=" + topic + " payload=" + payload));

        System.out.println("Topic snapshot: " + bus.topicSnapshot());

        Map<String, Object> orderCreated = new LinkedHashMap<>();
        orderCreated.put("order_id", "ORD-1001");
        orderCreated.put("user_id", "U-42");
        orderCreated.put("amount", 1499);

        int delivered = bus.publish("order.created", orderCreated);
        System.out.println("Delivered to " + delivered + " subscribers");

        Map<String, Object> userSignedUp = new LinkedHashMap<>();
        userSignedUp.put("user_id", "U-99");
        userSignedUp.put("plan", "free");
        delivered = bus.publish("user.signed_up", userSignedUp);
```

```
        System.out.println("Delivered to " + delivered + " subscribers");
    }
}
```